Programación de Comunicaciones en Red

Unidad de Trabajo 3

¿Qué es una red?

- Una red conecta dispositivos para compartir información.

- **Ejemplos:**
  - Redes locales (LAN), redes globales (WAN).

  - Dispositivos conectados: computadoras, servidores, routers.

Sistemas distribuidos

- Muchos sistemas computacionales de la actualidad siguen el modelo de computación distribuida.

  Aplicaciones a través de Internet, móviles, etc.

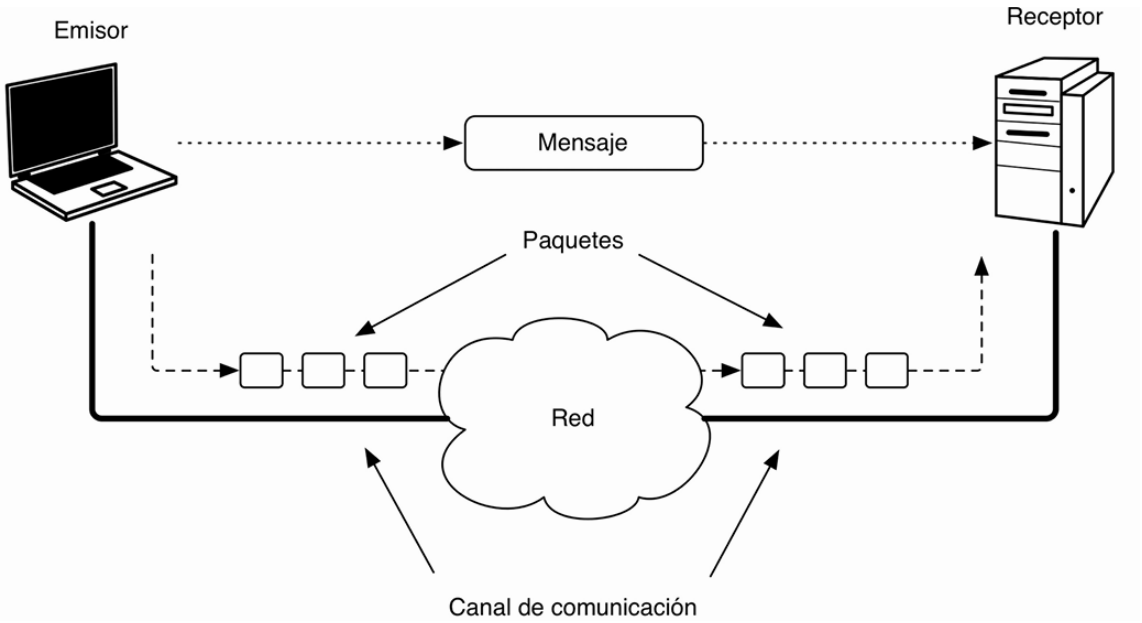
- La mayoría de los superordenadores modernos son sistemas distribuidos.

  Sistema distribuido: Está formado por más de un elemento computacional distinto e independiente

- Los elementos que forman el sistema están conectados a una red de comunicaciones.

Elementos del Proceso de comunicación

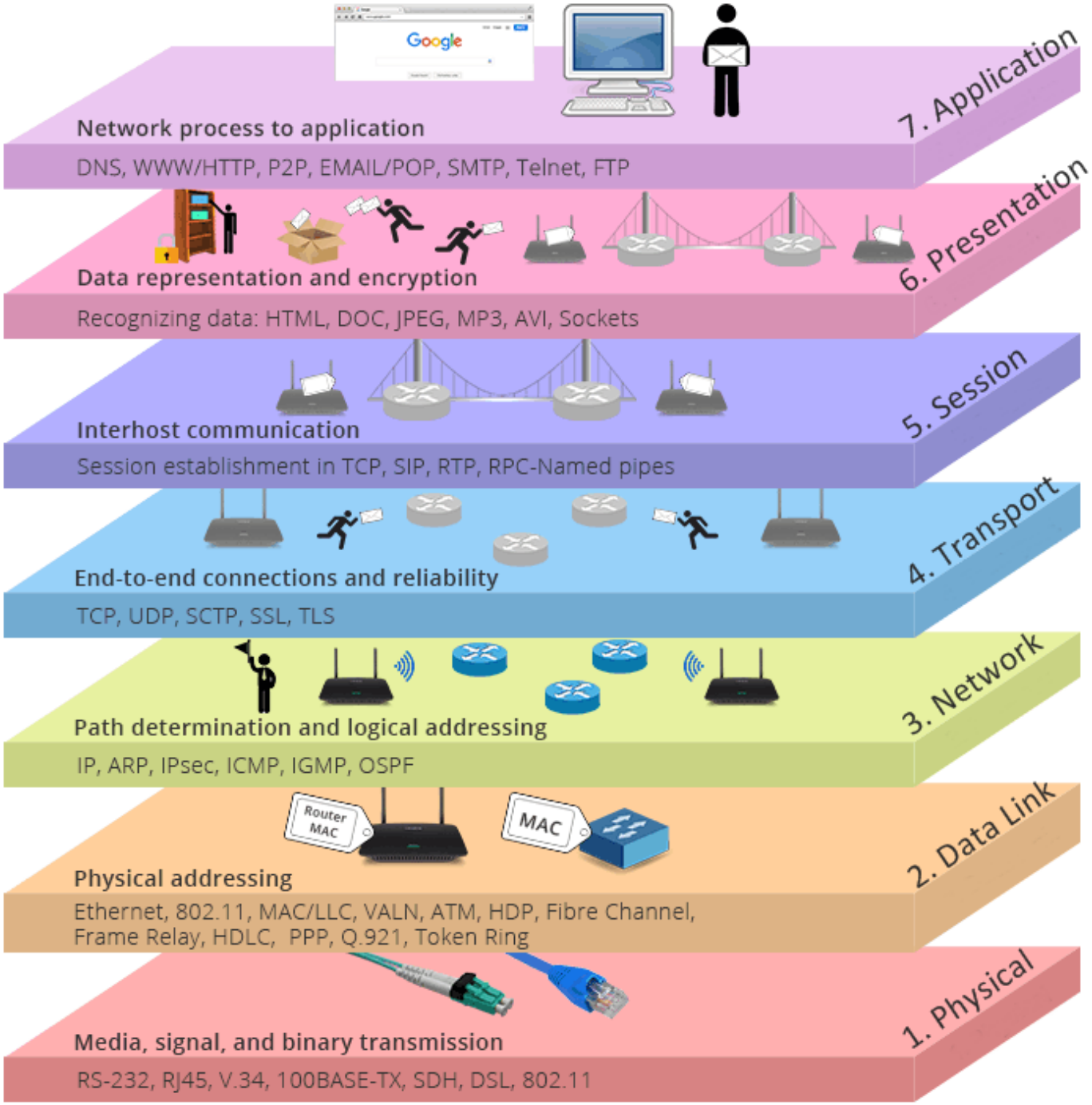
- Mensaje
- Emisor
- Receptor
- Unidad de datos
- Canal de comunicación
- Protocolo de comunicaciones



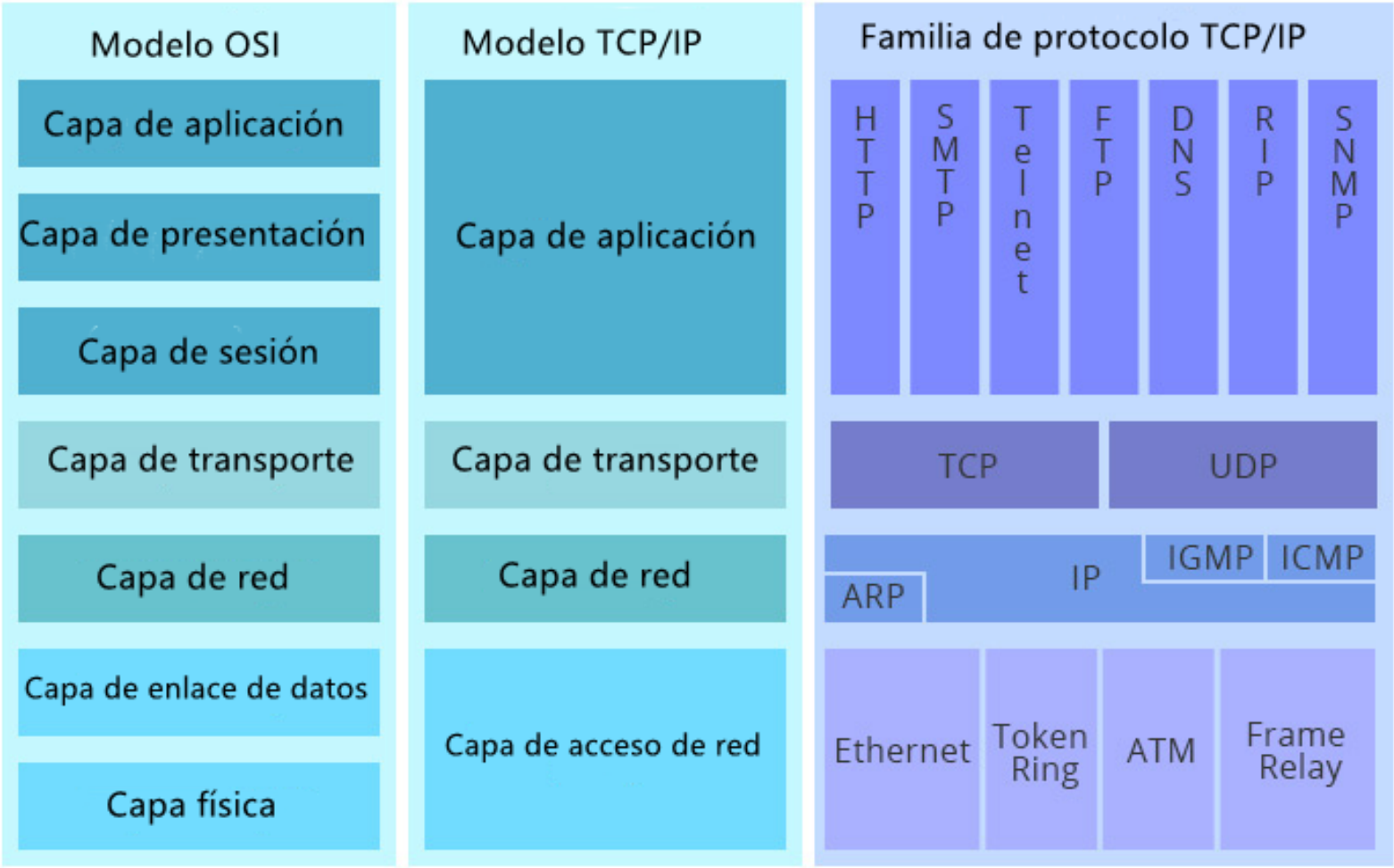
Protocolos de red

- Para que las diferentes aplicaciones que forman un sistema distribuido puedan comunicarse, debe existir una serie de mecanismos que lo hagan posible.
- Elementos HW y SW.
- Estos componentes se organizan en una jerarquía o pila de protocolos.
- Diferentes estándares (OSI y TCP/IP son los más relevantes).

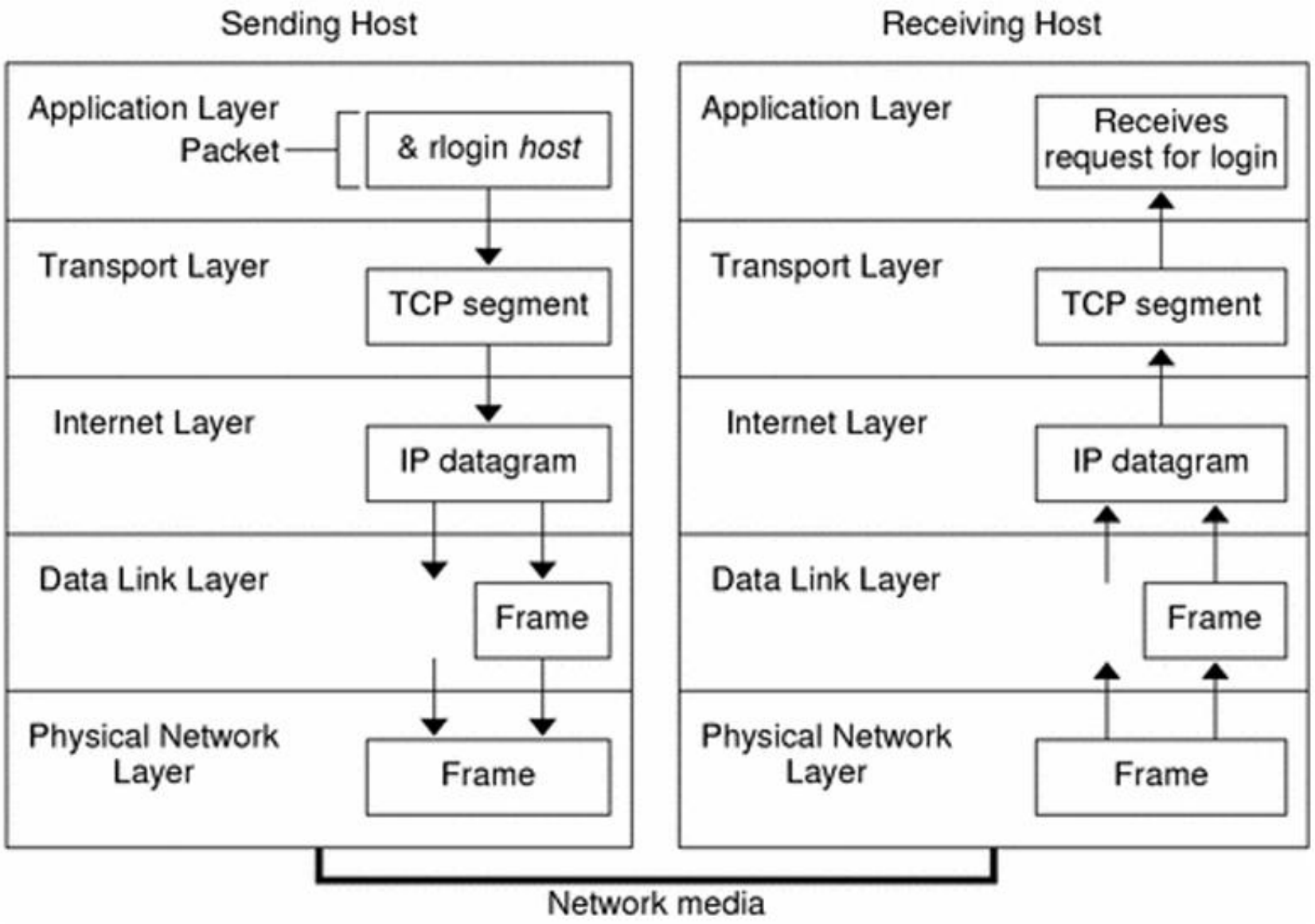
Modelo OSI


Google
7. Application
Network process to application
DNS, WWW/HTTP, P2P, EMAIL/POP, SMTP, Telnet, FTP
6. Presentation
Data representation and encryption
Recognizing data: HTML, DOC, JPEG, MP3, AVI, Sockets
5. Session
Interhost communication
Session establishment in TCP, SIP, RTP, RPC-Named pipes
4. Transport
End-to-end connections and reliability
TCP, UDP, SCTP, SSL, TLS
3. Network
Path determination and logical addressing
IP, ARP, IPsec, ICMP, IGMP, OSPF
Router MAC
MAC
2. Data Link
Physical addressing
Ethernet, 802.11, MAC/LLC, VALN, ATM, HDP, Fibre Channel,
Frame Relay, HDLC, PPP, Q.921, Token Ring
1. Physical
Media, signal, and binary transmission
RS-232, RJ45, V.34, 100BASE-TX, SDH, DSL, 802.11

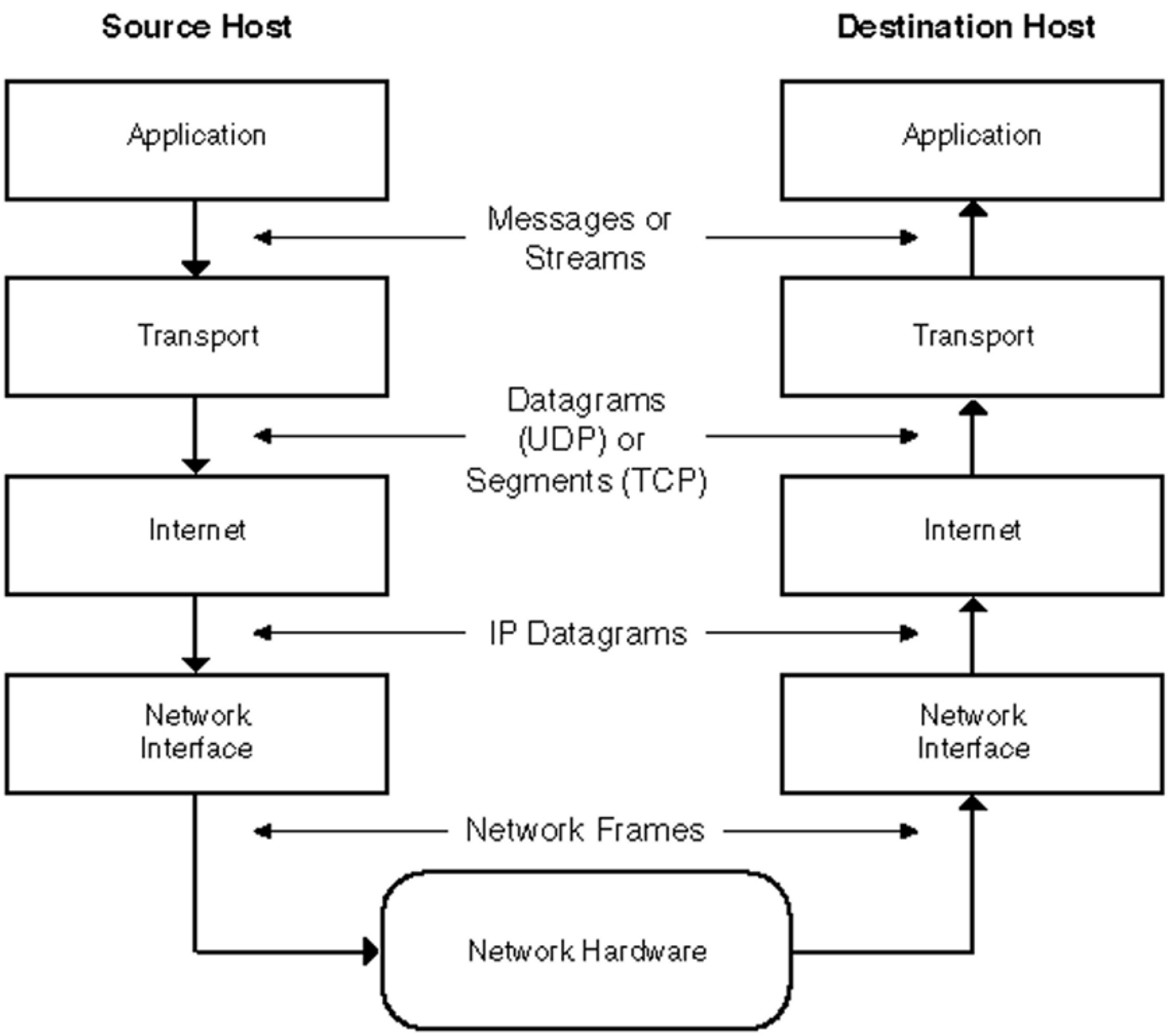
Modelo TCP/IP



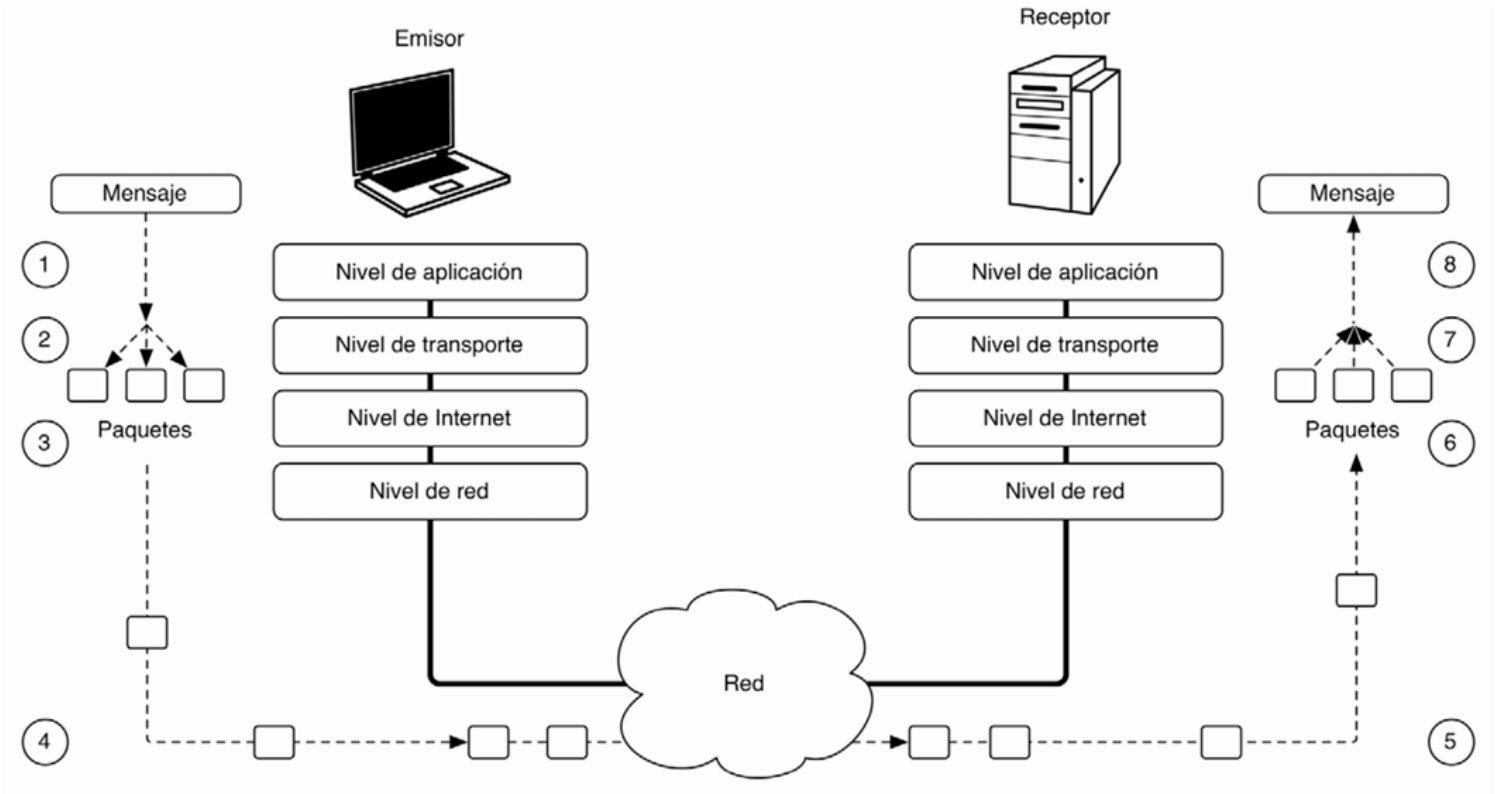
Unidades de datos (PDUs)



Abstracción capas



Proceso de comunicación



Direccionamiento IPv6

Las direcciones Ipv4 son direcciones de 32-bits.

La dirección IP se agrupa en cuatro octetos o bytes (grupos de 8 bits)

Se representan usando el valor en notación decimal de cada uno de los bytes, separados por puntos.

El valor mínimo para cada octeto es 0 y el valor máximo es 255.

192.168.0.100
127.0.0.1
10.1.100.1

Direccionamiento IPv4

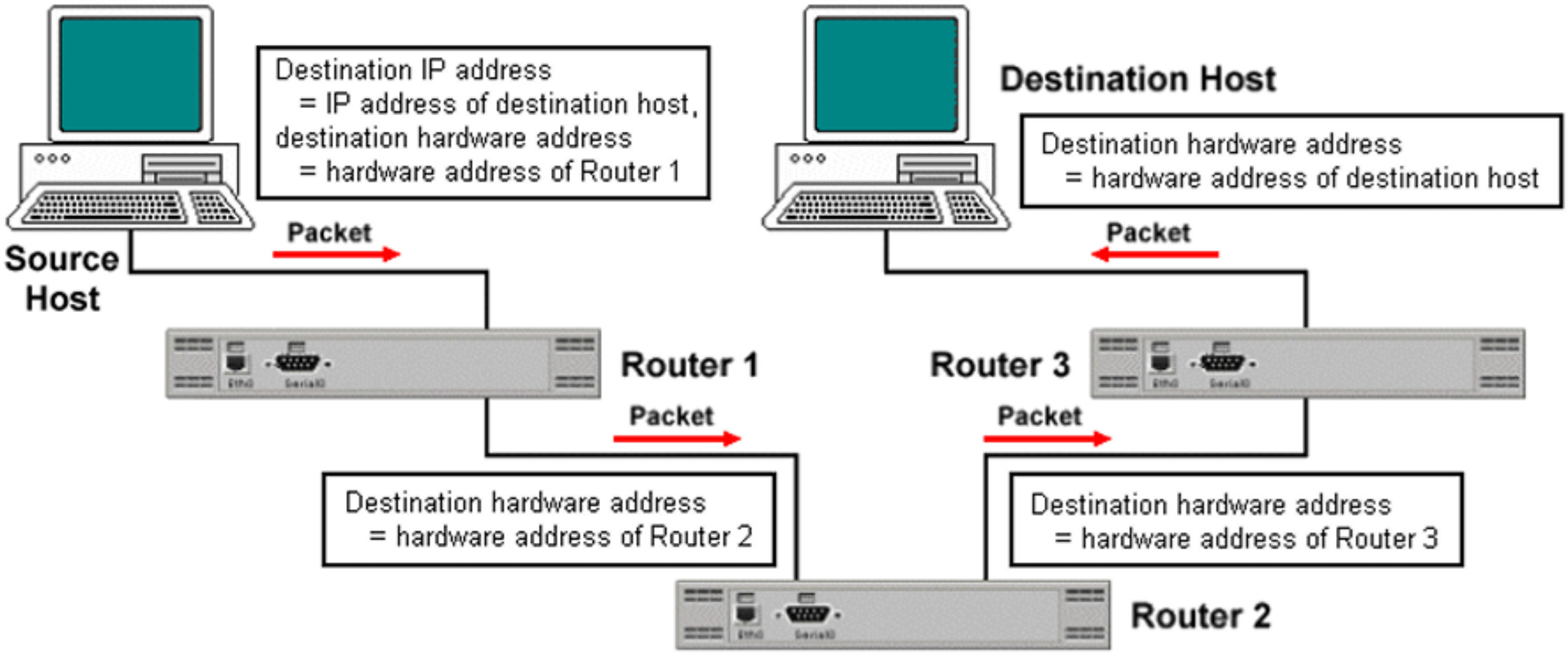
Las direcciones IPv6 está formadas por 64-bits para la dirección de red o prefijo de red, y otros 64 bits para el número de host.

Las direcciones IPv6 se escriben como 8 grupos de 4 dígitos hexadecimales separados por el caracter ':'.

Un grupo que sólo tiene ceros puede ser omitido. Los ceros iniciales también se pueden omitir.

Todas estas direcciones serían equivalentes.

2001:0db8:0000:0000:0000:0000:1428:57ab
2001:0db8:0000:0000:0000::1428:57ab
2001:0db8:0:0:0:0:1428:57ab 2001:0db8:0:0::1428:57ab
2001:0db8::1428:57ab 2001:db8::1428:57ab


Destination IP address
= IP address of destination host,
destination hardware address
= hardware address of Router 1
Destination Host
Destination hardware address
= hardware address of destination host
Source Host
Packet
Packet
Router 1
Router 3
Packet
Packet
Destination hardware address
= hardware address of Router 2
Destination hardware address
= hardware address of Router 3
Router 2

Cliente-Servidor

- **Cliente:**
  - Solicita servicios (navegador, cliente de correo).
- **Servidor:**
  - Proporciona servicios (servidor web, base de datos).

Clases Fundamentales en Java

- **InetAddress:** Información sobre direcciones IP y nombres de host.
- **NetworkInterface**: Información sobre las interfaces de red disponibles.
- **Socket:** Comunicación cliente-servidor (TCP).
- **ServerSocket:** Escucha conexiones entrantes.
- **DatagramSocket y DatagramPacket:** Comunicación UDP.

¿Qué es InetAddress?

- Proporciona información sobre direcciones IP y hosts.
- Métodos clave:
  - **getLocalHost():** Información del host local.
  - **getByName(String host):** Dirección IP de un nombre de host.
  - **getHostName():** Nombre del host.
  - **getHostAddress():** Dirección IP en formato String.

Información del host local

```
import java.net.*;

public class HostInfo {
	public static void main(String[] args) {
		try {
			InetAddress localHost = InetAddress.getLocalHost();
			System.out.println("Nombre del host local: " + localHost.getHostName());
			System.out.println("Dirección IP local: " + localHost.getHostAddress());
		} catch (UnknownHostException e) {
			System.out.println("Error al obtener la información del host.");
		}
	}
}
```

Información de un host remoto

```
import java.net.*;

public class HostInfo {
	public static void main(String[] args) {
		try {
			InetAddress google = InetAddress.getByName("www.google.com");
			System.out.println("Host remoto: " + google.getHostName());
			System.out.println("IP remoto: " + google.getHostAddress());
		} catch (UnknownHostException e) {
			System.out.println("Error al obtener la información del host.");
		}
	}
}
```

- El loopback (interfaz lo):
  - Es una subred 127.0.0.0/8.
  - Cualquier IP en el rango 127.0.0.1 a 127.255.255.254 resuelve al localhost y funcionan igual.

- En algunas distros Linux se configuran los siguientes hosts:

```
goya@goya-VirtualBox:~$ cat /etc/hosts
127.0.0.1       localhost
127.0.1.1       goya-VirtualBox
```

- Motivo:
  - DHCP puede tardar en asignar IP real.
  - Instalar paquetes requiere que hostname resuelva.
  - Así se evita que aplicaciones fallen si el host no tiene IP externa aún.

¿Por qué me resuelve localhost a 127.0.1.1?

¿Qué es una Interfaz de Red?

Una interfaz de red es un punto de conexión entre el sistema y la red (física o virtual).

Interfaz física: Ejemplo, Ethernet (cableada) o Wi-Fi (inalámbrica).

Interfaz virtual: Ejemplo, conexiones VPN o adaptadores de red creados por software

Cada interfaz puede tener asociada una o más direcciones IP, dependiendo de su configuración y del tipo de red a la que esté conectada.

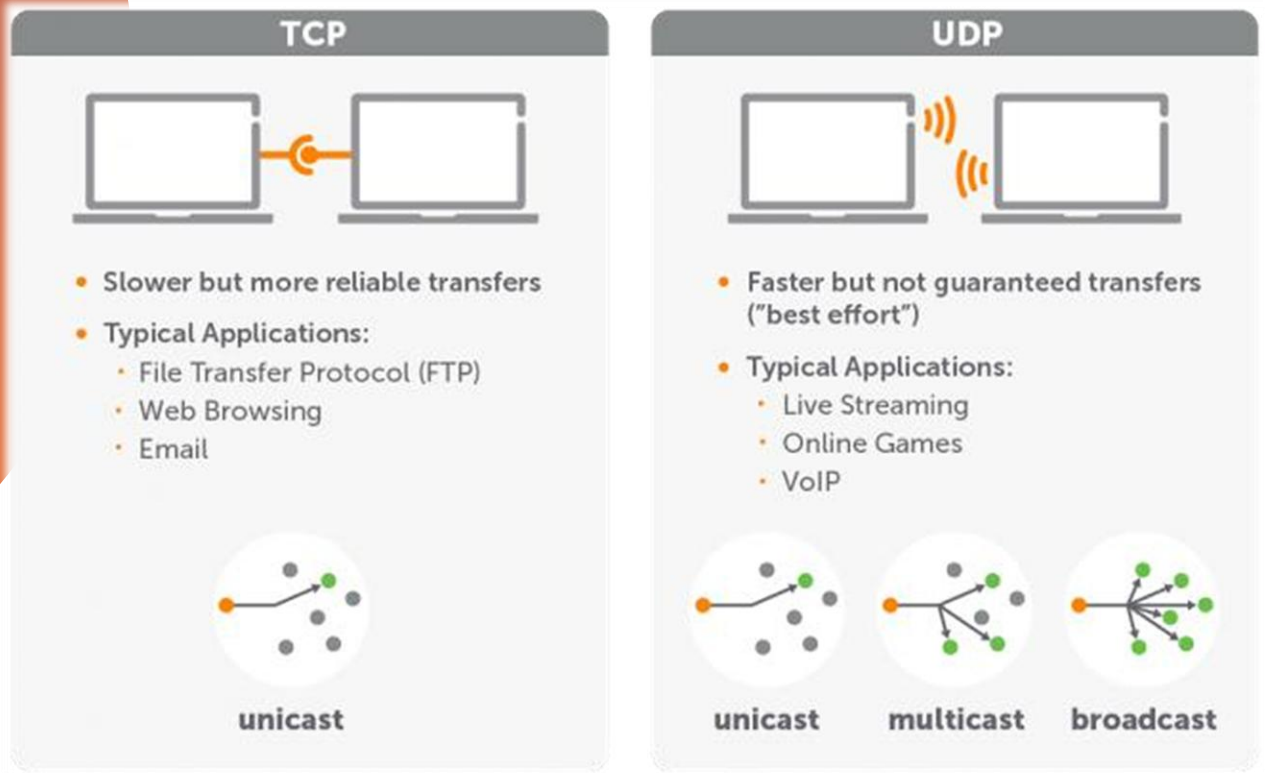
¿Qué es NetworkInterface?

- Proporciona información sobre interfaces de red.
- Métodos clave:
  - **getNetworkInterfaces():** Enumerar todas las interfaces del sistema.
  - **getName():** Nombre de la interfaz.
  - **getInetAddresses():** Enumeración con las direcciones IP asociadas a una interfaz.

```
Enumeration<NetworkInterface> interfaces = NetworkInterface.getNetworkInterfaces();

while (interfaces.hasMoreElements()) {
            NetworkInterface ni = interfaces.nextElement();

        System.out.println("Nombre de la interfaz: " + ni.getName());
        System.out.println("Interfaz activa: " + ni.isUp());
        System.out.println("Multicast soportado: " + ni.supportsMulticast());
        System.out.println("MAC Address: " + Arrays.toString(ni.getHardwareAddress()));

        // Obtener las direcciones IP asociadas
        Enumeration<InetAddress> direcciones = ni.getInetAddresses();
        while (direcciones.hasMoreElements()) {
                InetAddress ip = direcciones.nextElement();
                System.out.println("  Dirección IP: " + ip.getHostAddress());
                if (ip instanceof Inet4Address) {
                        System.out.println("    Tipo: IPv4");
                } else if (ip instanceof Inet6Address) {
                        System.out.println("    Tipo: IPv6");
                }
          }
}
```

Protocolos TCP y UDP

- **TCP (Transmission Control Protocol):**
  - Fiable y orientado a conexión.
  - Entrega en orden garantizada.
- **UDP (User Datagram Protocol):**
  - Sin conexión, rápido pero menos fiable.



Aspecto	TCP	UDP
Conexión	Orientado a conexión	Sin conexión
Fiabilidad	Entrega garantizada	No garantiza la entrega
Velocidad	Más lento	Más rápido
Ordenación	Paquetes ordenados	Desordenados
Aplicaciones	Transferencia de archivos	Streaming, juegos en línea

Protocolos TCP y UDP

Puertos

- Cuando una aplicación que se está ejecutando en un equipo quierese identifica comunicarse con otra aplicación de otro equipo, a sí misma con un **número de 16 bits**, que denominamos puerto.

- Ese **identificador** es usado por los protocolos de la capa de transporte (TCP o UDP) **para entregar los mensajes a la aplicación correcta dentro del equipo**.

- Los puertos van de 0 a 65535 (2^{16}), y se agrupan en tres rangos.

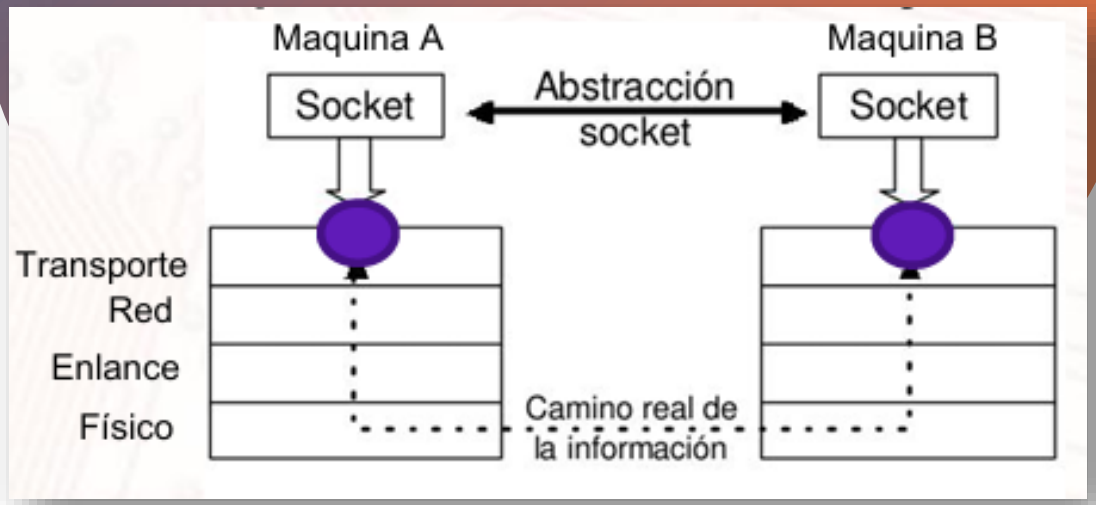
Puertos

Grupo de puertos	Rango de puertos	Descripción
Puertos bien conocidos o puertos del sistema	0 - 1023	Los usan los protocolos estándar y los servicios del SO
Puertos registrados	1024-49151	Reservados por empresas y organizaciones para sus propios servicios
Puertos efímeros	49152 - 65535	De libre disposición y uso para aplicaciones cliente y servidor

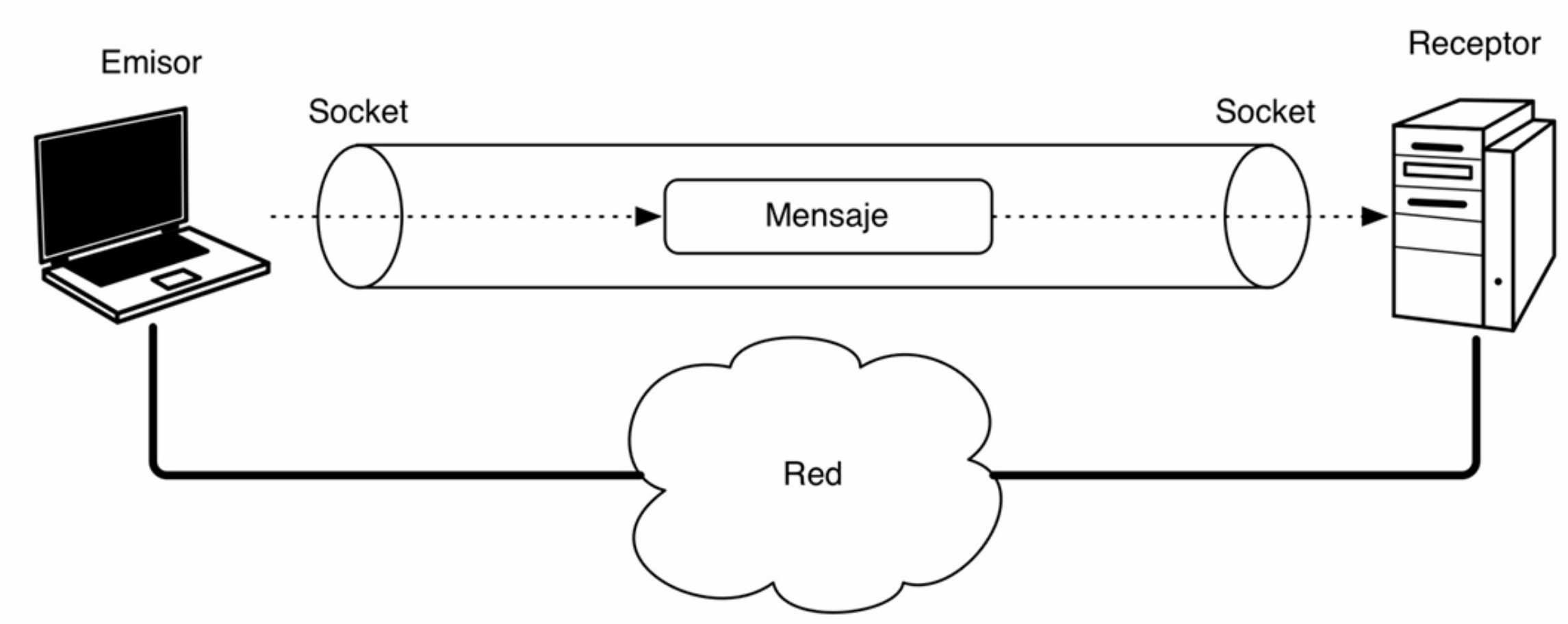
Puertos

- Los servidores de protocolos estándar como Telnet y FTP usan uno o más de estos puertos bien conocidos.
- Permite a aplicaciones cliente comunicarse con el servidor sin una petición previa para determinar el puerto en uso.
- Las aplicaciones cliente (navegadores, clientes de correo, etc) no necesitan usar puertos bien conocidos, al iniciar ellas la comunicación.
- A los procesos cliente se les asigna un número de puerto de forma dinámica por parte del SO, incluido en todos los datagramas que intercambia con el servidor.

Sockets

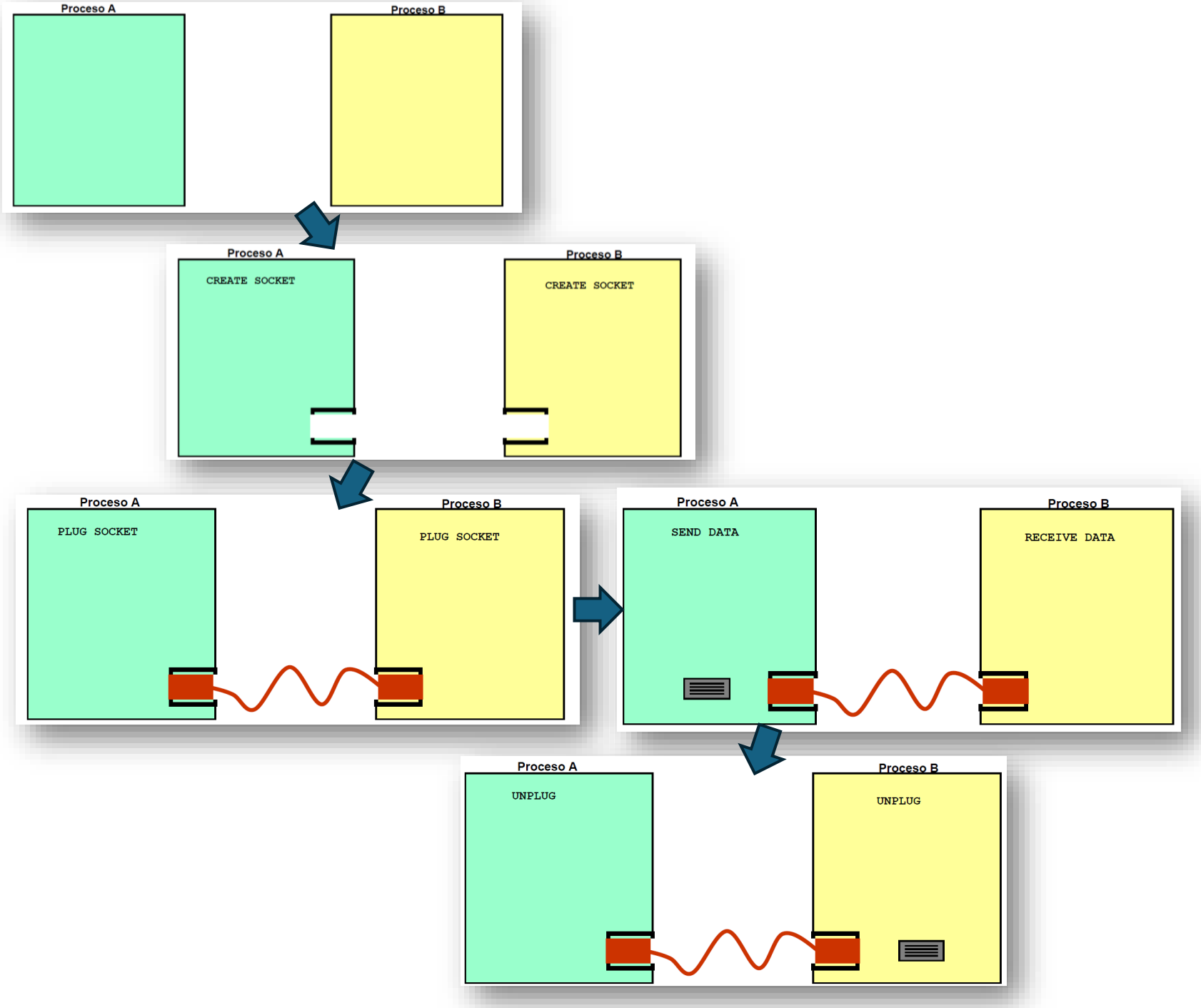


- Un socket es un punto de comunicación entre dos aplicaciones.
- Formado por combinación única de:
  - Dirección IP
  - Puerto
  - Protocolo de transporte
- Tipos:
  - **Socket TCP:** Conexión fiable.
  - **Socket UDP:** Rápido pero sin conexión.



Sockets

Sockets


Proceso A
Proceso B
Proceso A
CREATE SOCKET
Proceso B
CREATE SOCKET
Proceso A
PLUG SOCKET
Proceso B
PLUG SOCKET
Proceso A
SEND DATA
Proceso B
RECEIVE DATA
Proceso A
UNPLUG
Proceso B
UNPLUG

Sockets

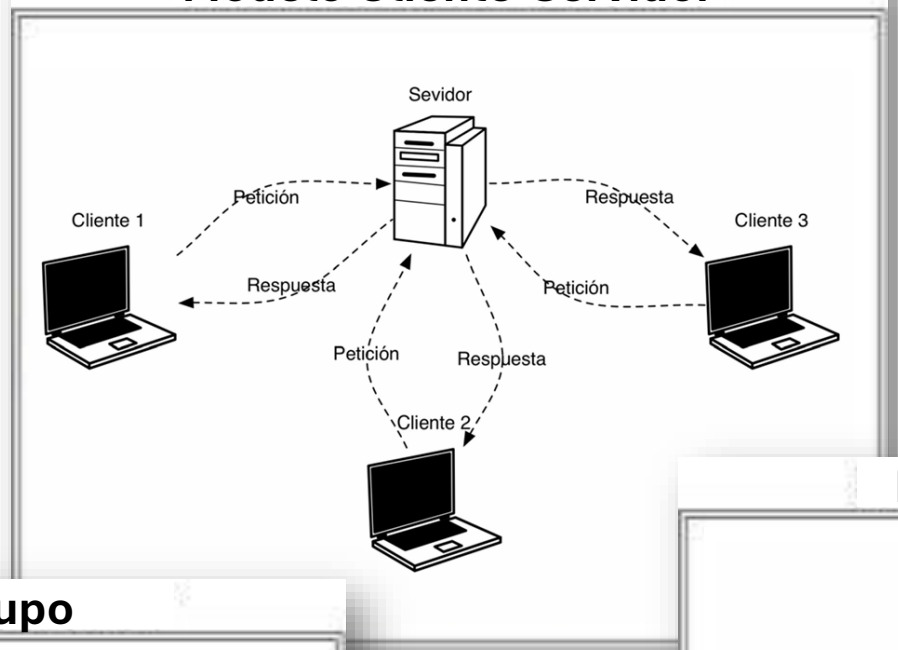
- La conexión usada por un cliente está formada por dos sockets, uno en el lado del cliente y otro en el lado del servidor.

- La conexión puede identificarse con una tupla:
  - Dirección IP de origen.
  - Dirección IP de destino.
  - Puerto de origen.
  - Puerto de destino.

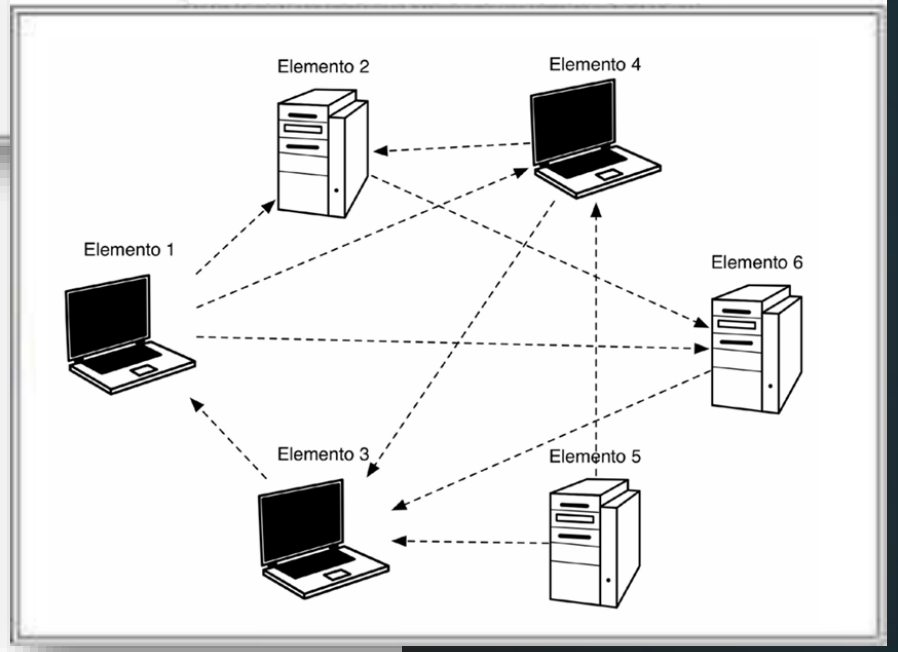
- Esto permite que múltiples aplicaciones cliente ejecutándose en máquinas diferentes puedan conectarse al mismo socket de destino en el servidor.

Modelos

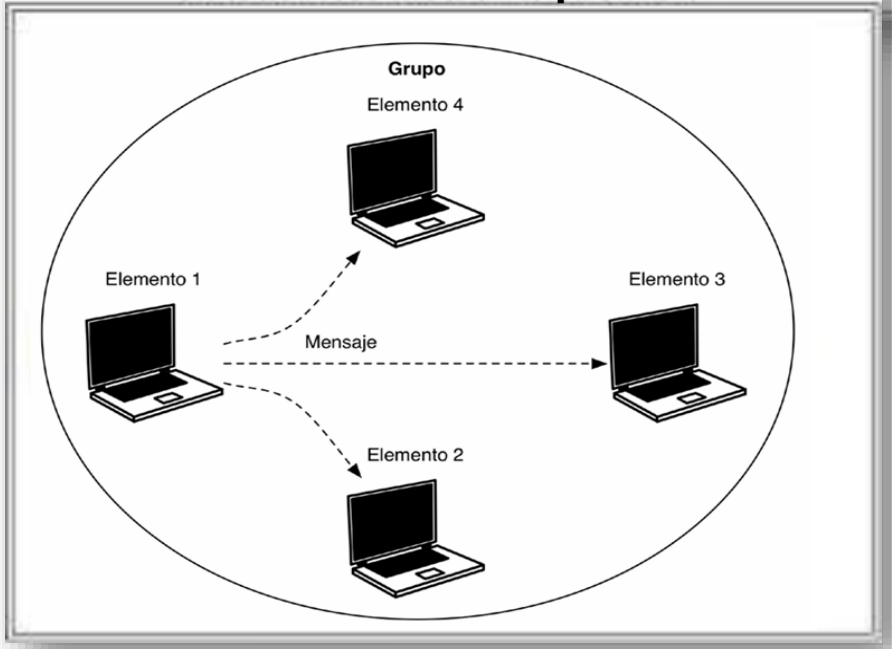
Modelo Cliente-Servidor



Modelo Peer-To-Peer

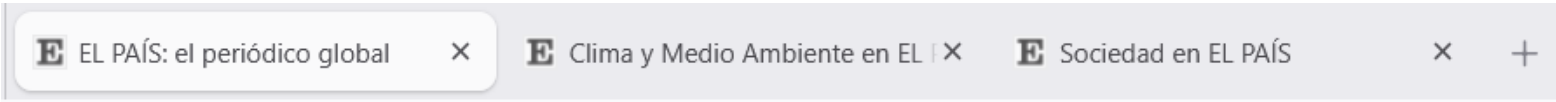


Modelo en Grupo



Sockets

- Usando sockets también es posible tener varias aplicaciones cliente corriendo en el mismo equipo y conectándose al mismo servidor (varias pestañas de un navegador).



WINDOWS: netstat -an | findstr "ESTABLISHED"

LINUX: netstat -ant | grep ESTABLISHED

- Hay aplicaciones corriendo dentro de un mismo sistema que utilizan los sockets para comunicarse consigo mismas.

```
C:\Users\[redacted]>netstat -n

Conexiones activas

  Proto  Dirección local        Dirección remota       Estado
  TCP    127.0.0.1:49679        127.0.0.1:49680        ESTABLISHED
  TCP    127.0.0.1:49680        127.0.0.1:49679        ESTABLISHED
  TCP    127.0.0.1:49681        127.0.0.1:49682        ESTABLISHED
  TCP    127.0.0.1:49682        127.0.0.1:49681        ESTABLISHED
  TCP    127.0.0.1:49715        127.0.0.1:49716        ESTABLISHED
  TCP    127.0.0.1:49716        127.0.0.1:49715        ESTABLISHED
```

Sockets TCP

Servidor:

- Crea un **ServerSocket** para escuchar conexiones entrantes.
- Acepta una conexión de un cliente utilizando el método **accept**().
- Recibe datos desde el cliente utilizando **InputStream**.

Cliente:

- Crea un **Socket** para conectarse al servidor.
- Envía datos utilizando **OutputStream**.

ServerSockets (TCP)

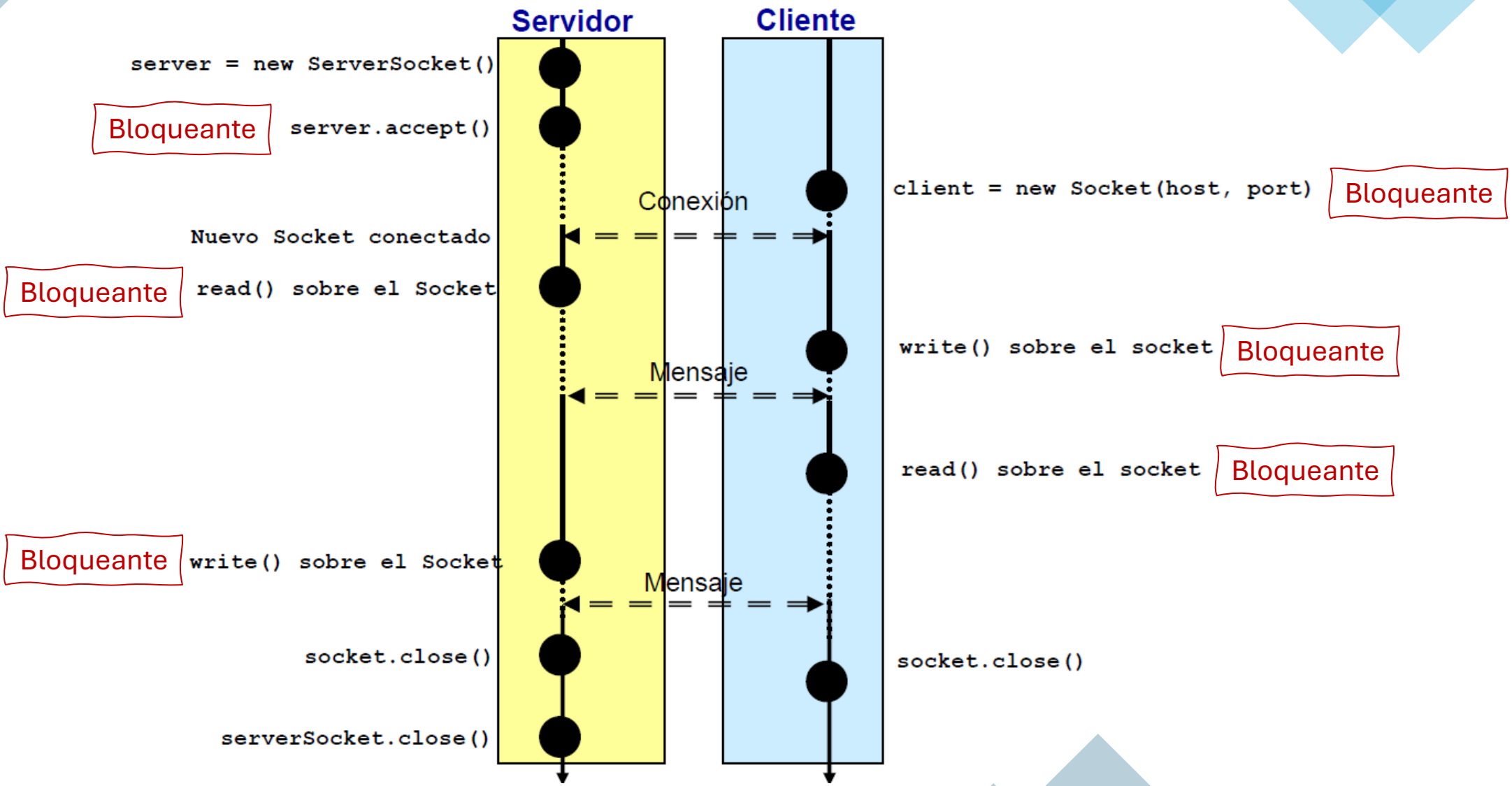
Método constructor	Utilidad
ServerSocket()	Crea un nuevo socket de conexión no atado a ningún puerto. Requiere hacer un bind a un puerto/interfaz.
ServerSocket(int port)	Crea un nuevo socket de conexión y lo ata (bind) al puerto especificado. El número de puerto debe estar comprendido entre 0 y 65535 inclusive. Si port es 0, se toma un puerto libre “al azar”.
ServerSocket(int port, int backlog)	Permite además especificar el tamaño de la cola de solicitudes de conexión no atendidas. Si el número de solicitudes no atendidas supera el tamaño especificado se rechazan solicitudes de conexión.
ServerSocket(int port, int backlog, InetAddress bindAddr)	Permite además especificar la dirección IP en la que se atenderán las solicitudes de conexión (para ordenadores que dispongan de varias). Si bindAddres null, el ServerSocket atenderá solicitudes de conexión en todas las interfaces.

Sockets (TCP)

Método constructor	Utilidad
Socket()	Crea un nuevo socket sin conectar.
Socket(InetAddress address, int port)	Crea un nuevo socket y lo conecta automáticamente con un socket remoto en la dirección IP y puertos especificados.
Socket(InetAddress address, int port, InetAddress localAddr, int localPort)	Permite además especificar la dirección IP local y el puerto local a los que se asociará el socket.
Socket(String host, int port)	El host remoto se puede especificar a través de su nombre de dominio. Se utilizarán los mecanismos de resolución del sistema operativo subyacente.

Conexión TCP cliente/servidor Sockets en Java

Sincronización de cliente y servidor



Ejemplo práctico 1

- Vamos a programar:
  - Un servidor que escuche permanentemente por el puerto TCP/5000.
  - Un cliente que se conecte contra él.
  - Cada vez que un cliente se conecte, el servidor le informará de qué número de cliente le ha asignado.

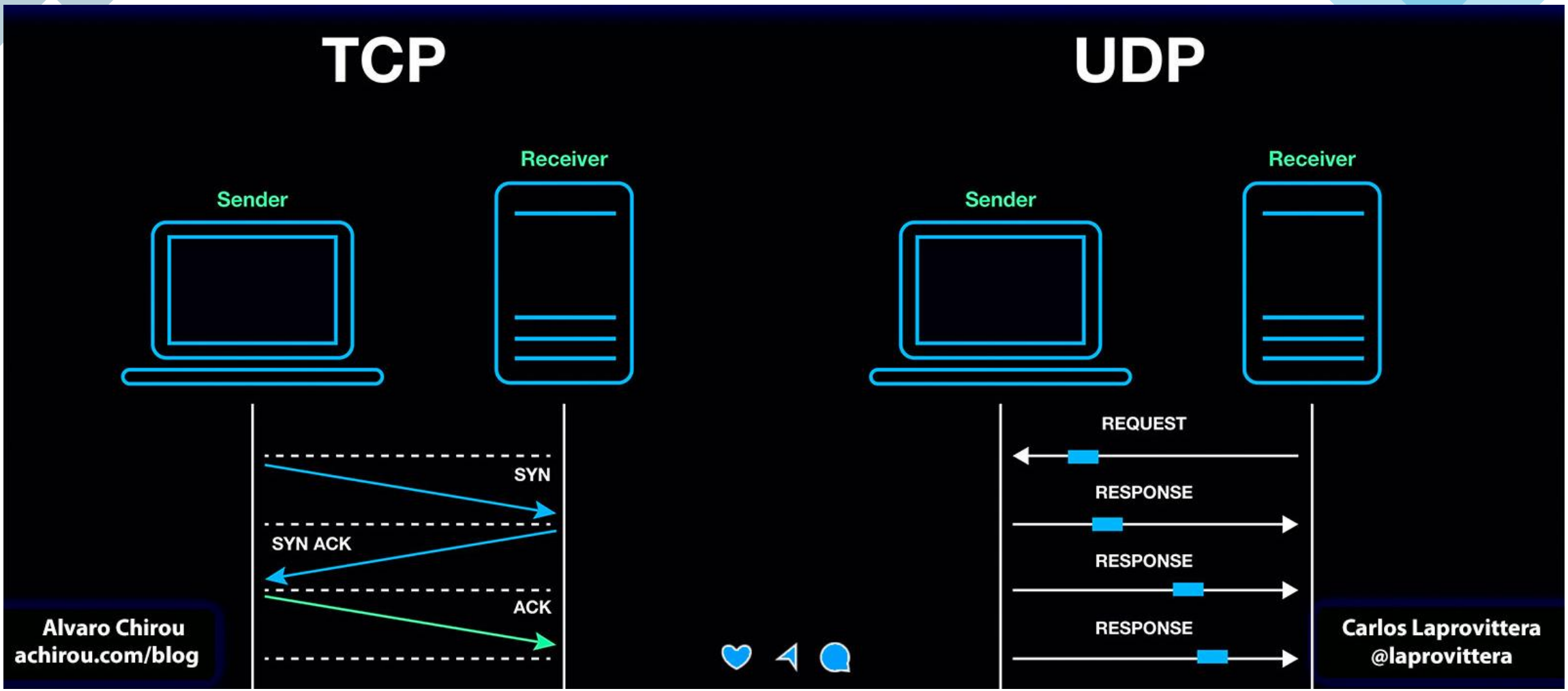
Sockets UDP

Servidor:

- Crea un **DatagramSocket** para escuchar conexiones entrantes.
- Acepta una conexión de un cliente utilizando el método **receive**().
- Recibe datos desde el cliente utilizando **DatagramPacket** y un buffer de bytes.

Cliente:

- Crea un **DatagramSocket** para conectarse al servidor.
- Envía datos utilizando **DatagramPacket** y un buffer de bytes.


TCP
Sender
Receiver
SYN
SYN ACK
ACK
Alvaro Chirou
achirou.com/blog
UDP
Sender
Receiver
REQUEST
RESPONSE
RESPONSE
RESPONSE
Carlos Laprovittera
@laprovittera

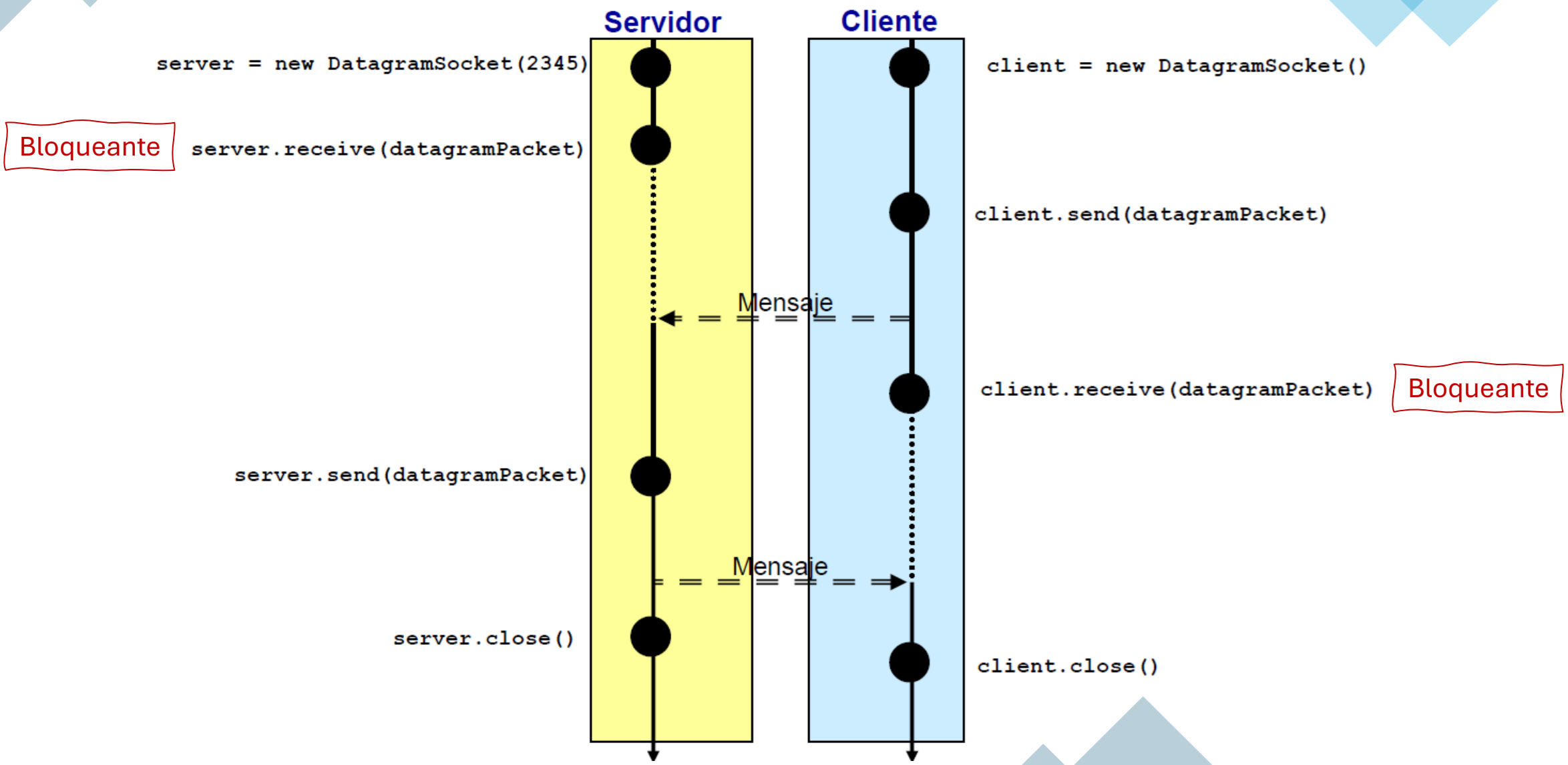
Datagramas (UDP)

Método constructor	Utilidad
DatagramSocket()	Crea un socket de datagrama y lo ata a un puerto libre en la máquina local. Suele usarse para desarrollar el extremo cliente de una aplicación.
DatagramSocket(int port)	Crea un nuevo socket de datagrama y lo ata al puerto especificado en todas las interfaces de la máquina local. Este constructor suele utilizarse para desarrollar el extremo servidor de una aplicación.
DatagramSocket(int port, InetAddress laddr)	Crea un nuevo socket de datagrama y lo ata al puerto especificado en la dirección IP dada sobre la máquina local (para máquinas con varias interfaces). Este constructor suele utilizarse para crear el extremo servidor de una aplicación.
DatagramSocket(SocketAddress bindaddr)	Crea un nuevo socket de datagrama y lo ata a la dirección especificada (puerto y dirección IP). Este constructor suele utilizarse para crear el extremo servidor de una aplicación.

Datagramas (UDP)

Método constructor	Utilidad
DatagramPacket(byte[] buf, int length)	Construye un DatagamPacket para recibir datagramas con hasta length bytes de datos.
DatagramPacket(byte[] buf, int length, InetAddress addr, int port)	Crea un nuevo DatagramPacket para el envío de los datos especificados a la dirección y puerto remotos dados.
DatagramPacket(byte[] buf, int offset, int length)	Construye un DatagramPacket para la recepción de datos, que se depositarán con el offset especificado en el buffer dado.
DatagramPacket(byte[] buf, int offset, int length, InetAddress addr, int port)	Crea un DatagramPacket para enviar un datagrama UPD que contenga length bytes de datos a partir de buf[offset].

Sincronización de los DatagramSocket



Ejemplo práctico 2

- Vamos a programar:
  - Un servidor que escuche permanentemente por el puerto UDP/6789 con un tamaño de buffer de 1000 bytes.
  - Un cliente que se conecte contra él.
  - Cada vez que un cliente se conecte, el servidor le informará de qué número de cliente le ha asignado.